261019 시퀀스검출기_moore_mealy_박수민



1. source code
[moore_fsm.v]

```
`timescale 1ns / 1ps

module moore_fsm(
    input clk,
    input reset,
    input en,
    input x,
    output reg y
);

    localparam [1:0]
        S0 = 2'b00,
        S1 = 2'b01,
        S2 = 2'b10,
        S3 = 2'b11;

    reg [1:0] r_curr_state, r_next_state;

    // 1. 현재 상태 저장
```

```
always @(posedge clk or posedge reset) begin
    if (reset)
        r_curr_state <= S0;
    else if (en)
        r_curr_state <= r_next_state;
end

// 2. 다음 상태 결정
always @(*) begin
    r_next_state = r_curr_state;

    case (r_curr_state)
        S0: begin
            if (x)
                r_next_state = S1;
            else
                r_next_state = S0;
        end

        S1: begin
            if (x)
                r_next_state = S1;
            else
                r_next_state = S2;
        end

        S2: begin
            if (x)
                r_next_state = S3;
            else
                r_next_state = S0;
        end

        S3: begin
            if (x)
                r_next_state = S1;
            else
                r_next_state = S2;
        end

        default:
```

```
                r_next_state = S0;
        endcase
    end

    // 3. Moore 출력
    // 현재 상태가 S3이면 y = 1
    always @(*) begin
        if (r_curr_state == S3)
            y = 1'b1;
        else
            y = 1'b0;
    end

endmodule
```

[mealy_fsm.v]

```
`timescale 1ns / 1ps

module mealy_fsm(
    input clk,
    input reset,
    input en,
    input x,
    output reg y
);

    localparam [1:0]
        S0 = 2'b00,
        S1 = 2'b01,
        S2 = 2'b10;

    reg [1:0] r_curr_state, r_next_state;

    // 현재 상태 저장
    always @(posedge clk or posedge reset) begin
        if (reset)
            r_curr_state <= S0;
        else if (en)
            r_curr_state <= r_next_state;
    end
```

```
    // 다음 상태 및 출력 결정
    always @(*) begin
        r_next_state = r_curr_state;
        y = 1'b0;

        case (r_curr_state)
            S0: begin
                if (x)
                    r_next_state = S1;
            end

            S1: begin
                if (!x)
                    r_next_state = S2;
            end

            S2: begin
                if (x) begin
                    y = 1'b1;
                    r_next_state = S1;
                end
                else
                    r_next_state = S0;
            end

            default:
                r_next_state = S0;
        endcase
    end

endmodule
```

[top_fsm_101_basys3.v]

```
`timescale 1ns / 1ps

module top_fsm_101_basys3(
    input clk,              // W5 : 100MHz
    input reset,            // R2 : SW0
    input btnU,              // T18: BTNU = 1
```

```
    input btnD,             // U17: BTND = 0
    output reg led,         // U16: LD0 Mealy
    output reg led_moore // E19: LD1 Moore
);

    wire step_u;
    wire step_d;

    wire step;
    wire x;

    wire mealy_y;
    wire moore_y;

    // BTNU: 1 입력 버튼
    button_step u_btnU(
        .clk(clk),
        .reset(reset),
        .btn(btnU),
        .step(step_u)
    );

    // BTND: 0 입력 버튼
    button_step u_btnD(
        .clk(clk),
        .reset(reset),
        .btn(btnD),
        .step(step_d)
    );

    // 버튼 두 개를 동시에 누르면 입력 무시
    assign step = step_u ^ step_d;

    // BTNU = 1, BTND = 0
    assign x = step_u & ~step_d;

    // Mealy FSM
    mealy_fsm u_mealy(
        .clk(clk),
        .reset(reset),
        .en(step),
```

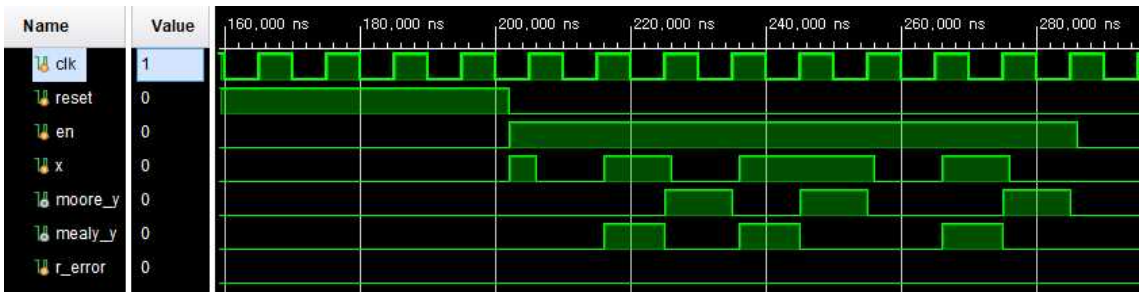
```
        .x(x),
        .y(mealy_y)
    );

    // Moore FSM
    moore_fsm u_moore(
        .clk(clk),
        .reset(reset),
        .en(step),
        .x(x),
        .y(moore_y)
    );

    // 버튼 입력 시 출력을 LED에 저장
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            led <= 0;
            led_moore <= 0;
        end
        else if (step) begin
            led <= mealy_y;
            led_moore <= moore_y;
        end
    end

endmodule
```

2. simulation



입력: 101011010
moore 000101001
mealy 001010010